

Linguagem de Programação I

Prof. Orlando Saraiva Júnior
orlando.nascimento@cps.sp.gov.br

Repetições

Repetições representam a base de vários programas. São utilizadas para executar a mesma parte do programa várias vezes, normalmente dependendo de uma condição.

Por exemplo, para imprimir três números na tela:

```
print(1)
```

```
print(2)
```

```
print(3)
```

Porém, se o objetivo fosse escrever 100 números, essa solução seria muito trabalhosa e repetitiva.

A estrutura de repetição aparece para nos auxiliar a resolver esse tipo de problema.

Em um programa de computador, a repetição também é chamada de **iteração**.

while

Uma das estruturas de repetição do python é o while, que repete um bloco enquanto a condição for verdadeira.

Seu formato é apresentado a seguir:

```
x = 1
```

```
while x <= 3:
```

```
    print(x)
```

```
    x = x + 1
```

Uma das estruturas de repetição do python é o while, que repete um **bloco** enquanto a condição for verdadeira.

x = 1

while x <= 3 :

print(x)

x = x + 1

Uma das estruturas de repetição do python é o while, que repete um bloco enquanto a condição for verdadeira.

```
x = 1
```

```
while x <= 3 :
```

```
    print(x)
```

```
    x = x + 1
```


Mais formalmente, aqui está o fluxo de execução para uma instrução while:

- 1) 1. Determine se a condição é verdadeira ou falsa.
- 2) 2. Se for falsa, saia da instrução while e continue a execução da próxima instrução.
- 3) 3. Se a condição for verdadeira, execute o corpo e então volte ao passo 1.

Este tipo de fluxo chama-se loop (laço), porque o terceiro passo faz um loop de volta ao topo.

O corpo do loop deve mudar o valor de uma ou mais variáveis para que, a certa altura, a condição fique falsa e o loop termine. Senão o loop vai se repetir para sempre, o que é chamado de ***loop infinito***.



Atividade

Peça um número N ao usuário.

Use `while` para contar de N até 0, imprimindo os números na tela e no final imprimir a mensagem "Liberado vôo".





Atividade

Realizar as três atividades do **Exercício de Fixação.**

- 1) Contagem Progressiva
- 2) Senha Correta
- 3) Soma de Números



Repetições: acumuladores e contadores

No programa abaixo, temos uma variável como acumulador e outra variável como contador:

n = 1

soma = 0

while n <= 10:

x = int(input(f'Digite o {n} número'))

soma = soma + x

n = n + 1

Repetições: acumuladores e contadores

No programa abaixo, temos uma variável como **acumulador** e outra variável como contador:

$n = 1$

soma = 0

while $n \leq 10$:

$x = \text{int}(\text{input}(f\text{'Digite o \{n\} número'}))$

soma = **soma** + x

$n = n + 1$

Repetições: acumuladores e contadores

No programa abaixo, temos uma variável como acumulador e outra variável como **contador**:

n = 1

soma = 0

while ***n*** <= 10:

x = int(input(f'Digite o {n} número'))

soma = *soma* + *x*

n = ***n*** + 1

Para simplificar, Python oferece instruções de atribuição especiais:

$n = 1$

$soma = 0$

$while\ n \leq 10:$

$x = int(input(f'Digite o \{n\} \text{número}'))$

$soma \ +=\ x$

$n \ +=\ 1$



Instruções de Atribuição Especiais

Operador	Exemplo	Equivalência
<code>+=</code>	<code>x += 1</code>	<code>x = x + 1</code>
<code>-=</code>	<code>y -= 1</code>	<code>y = y - 1</code>
<code>*=</code>	<code>c *= 2</code>	<code>c = c * 2</code>
<code>/=</code>	<code>d /= 2</code>	<code>d = d / 2</code>
<code>**=</code>	<code>e **= 2</code>	<code>e = e ** 2</code>
<code>//=</code>	<code>f //= 4</code>	<code>F = f // 4</code>

Embora seja muito útil, `while` só verifica sua condição de parada no início de cada repetição. Dependendo do problema, a capacidade de terminar o `while` dentro do bloco a repetir pode ser interessante.

Este é o papel da instrução **`break`**.

s = 0

while True:

v = int(input('Digite o número 0 para sair'))

If v == 0:

break

s += v

print(s)

Podemos combinar vários while de forma a obter resultados mais interessantes.

```
tabuada = 1
```

```
while tabuada <= 10:
```

```
    numero = 1
```

```
    while numero <= 10:
```

```
        print(f" {tabuada} x {numero} = {tabuada x numero}")
```

```
        numero += 1
```

```
    tabuada += 1
```

for

Em Python, o laço for é usado para percorrer elementos de sequências (como listas, strings e tuplas, assunto de próximas aulas) ou para iterar sobre um intervalo numérico (com uso do `range()`).

Diferente de linguagens como C++, onde o for precisa de uma variável de controle, condição e incremento, em Python o for é mais simples e direto.

O **range()** gera uma sequência de números dentro de um intervalo específico.

`range(fim)`: Gera números de 0 até `fim - 1`.

`range(início, fim)`: Gera números de `início` até `fim - 1`.

`range(início, fim, passo)`: Gera números de `início` até `fim - 1`, pulando de acordo com o `passo`.

```
for i in range(5):  
    print(i)
```

```
for i in range(1, 11):  
    print(i)
```

```
for i in range(0, 11, 2):  
    print(i)
```

Mais formalmente, aqui está o fluxo de execução para uma instrução for:

- 1) Inicialização: O for seleciona o primeiro valor da sequência ou intervalo (range).
- 2) Verificação: Se houver um próximo valor disponível, ele executa o corpo do laço.
- 3) Execução: O bloco de código dentro do for é executado com o valor atual.
- 4) Próximo Valor: O for avança para o próximo valor da sequência.
- 5) Retorno ao Passo 2: O loop continua até que não haja mais valores na sequência.
- 6) Fim do Laço: Quando não há mais valores, a execução do for termina e o programa segue para a próxima instrução.

Esse fluxo é chamado de loop iterativo, pois percorre uma sequência pré-definida de elementos.



For ou While ?

Python possui dois principais laços de repetição:

for → Quando sabemos quantas vezes vamos repetir.

while → Quando repetimos enquanto uma condição for verdadeira.

Ambos são usados para iterar sobre sequências ou executar blocos de código várias vezes.

Risco de loop infinito é baixo com **for**, visto que há um limite. Já com **while**, o risco é alto, visto que podemos esquecer de atualizar a condição de loop.

Diferente do **while**, o **for** já sabe quantas vezes vai repetir e não precisa de uma variável de controle explícita.



Fixação



Atividade

Realizar as atividades **Exercício de Fixação 2.**

- 1) Contagem Progressiva
- 2) Tabuada
- 3) Números Pares de 1 a 20



Dúvidas

Prof. Orlando Saraiva Júnior
orlando.nascimento@cps.sp.gov.br

Exercício de Fixação

Realize a lista de exercícios do dia.
Disponível no github do professor.



Tempo para Atividade
